

I. IDENTIFICACIÓN:

Curso: AUTOMATAS Y LENGUAJES	Año: 2026
Semestre: V	Carrera: Ing. Sistemas
Catedrático: Ing. Baudilio Boteo	E-mail: boteob@umes.edu.gt

II. OBJETIVO:

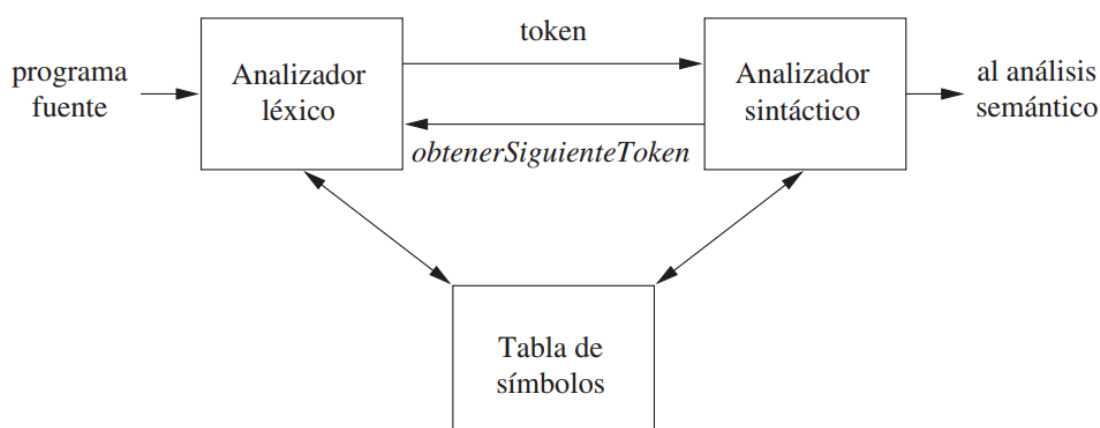
- Implementar un analizador léxico funcional que procese código fuente y genere una lista estructurada de tokens.
- Identificar lexemas válidos del lenguaje
- Clasificar tokens por tipo

III. DESCRIPCIÓN:

Analizador Léxico [MicroC]:

Consiste en implementar la funcionalidad del Analisis lexico al Compilador, siendo esta la primera etapa de un compilador completo como se muestra en la figura I, el objetivo será indicar si los símbolos pertenecen o no pertenecen al lenguaje analizado (en nuestro caso C++)

Figura I. Estructura de primeras etapas del Compilador



Para este propósito se debe realizar la siguiente estructura de clases y funciones:

Figura II. Diagrama UML de Compilador



Clase: Frame

Uso: Define los objetos para el frame o visualización gráfica del compilador con los botones mínimos para su funcionamiento

Funciones:

- OpcNuevo_Click()
- OpcAbrir_Click()
- OpcGuardar_Click()
- OpcGuardar Como_Click()
- OpcSalir_Click()

- OpcCompliar_Click()

Clase: AnalizadorLexico

Uso: Define las propiedades y funcionalidades de los objetos para el uso del analizador léxico

Funciones:

- Int **GetAlfabetoAlfanumerico** (char c)
- Int **GetAlfabetoNumero** (char c)
- Int **GetAlfabetoSimbolo** (char c)
- Void **IdentificadorPalabraReservada** (String Archivo)
- Void **EnteroReal** (String Archivo)
- Void **AutomataComentario** (String Archivo)
- List <String> **AnalisisLexico** (String Archivo)

Clase: UnidadesLexicas

Uso: Define las propiedades y funcionalidades de los objetos para el uso de la tabla de símbolos del lenguaje

Funciones:

- Int **GetTokenPalabra** (String Lexema)
- Int **GetTokenSimbolo** (String Lexema)

FASE I: Aplicar tareas Básicas

1. Generar Lista de tokens, que serán enviados al analizador sintáctico
2. Eliminar espacios en Blanco del código fuente
3. Eliminar tabuladores, saltos de línea y caracteres especiales del código fuente
4. Relacionar líneas de código con el análisis
5. Identificar lexemas simples y relacionar con su respectivo token

FASE II: Completar lista de Tokens

1. Identificar palabras reservadas
2. Identificar Numeros
3. Identificar comentarios

A continuación se adjunta diagrama de flujo para esta fase:

Figura III. Diagrama de Flujo compilador Fase 1 (parte 1)

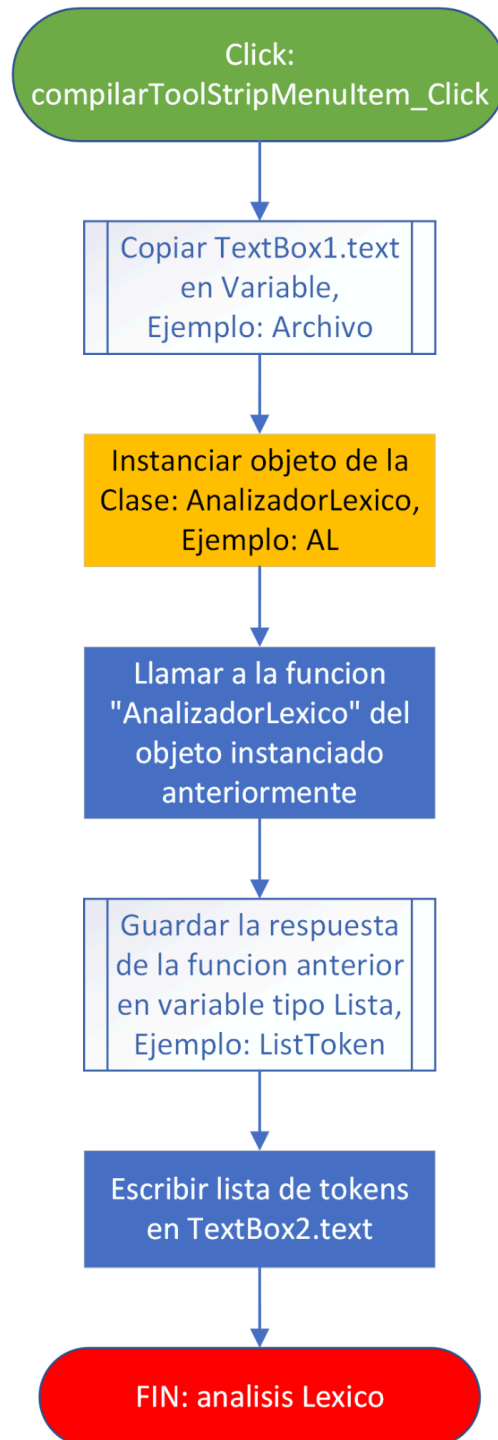


Figura IV. Diagrama de Flujo compilador Fase 1 (parte 2)

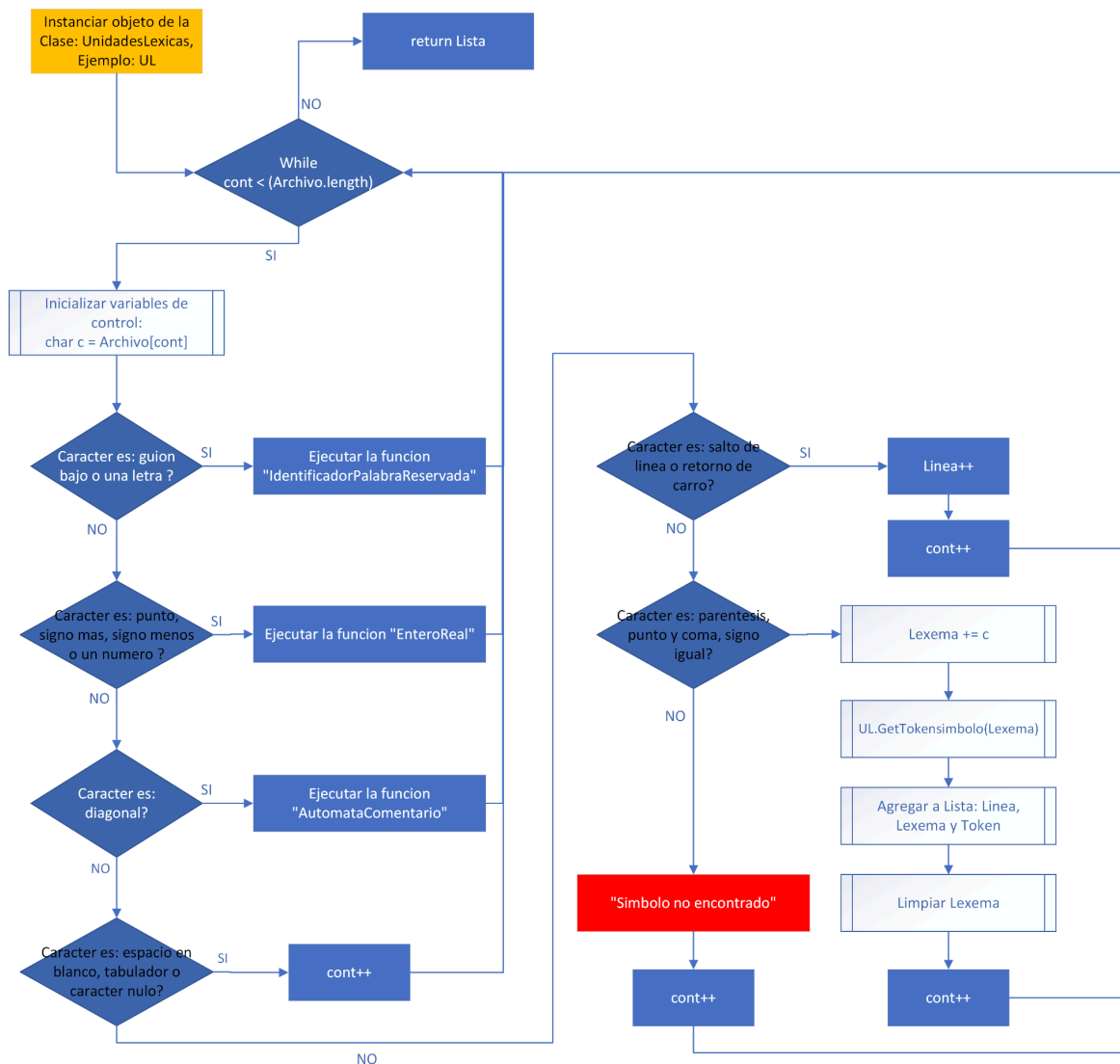


Figura V. Diagrama de Flujo compilador Fase 1 (parte 3)

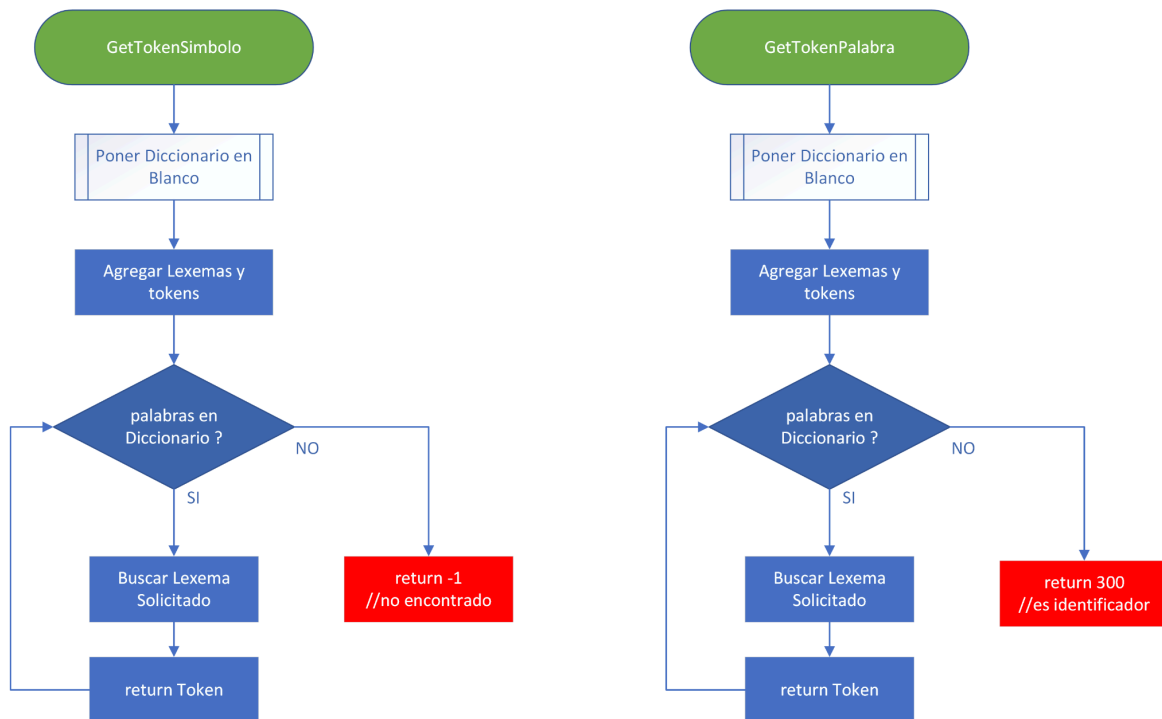
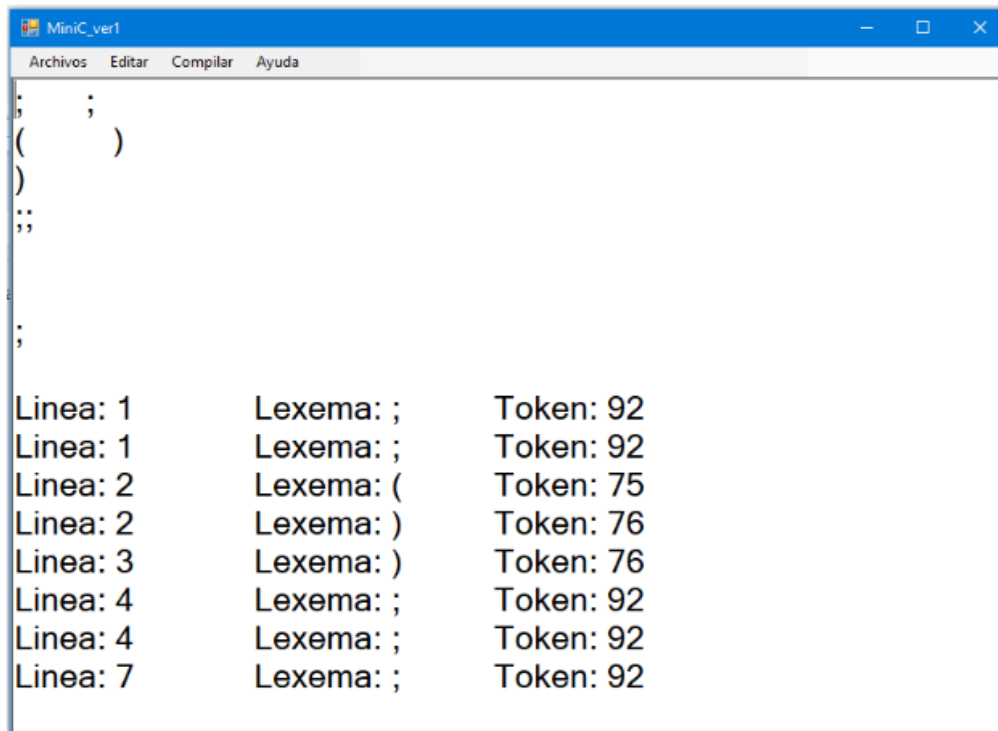


Figura V. Ejemplo y Resultado de la compilación



```

; ;
( )
)
;;
;

Linea: 1      Lexema: ;      Token: 92
Linea: 1      Lexema: ;      Token: 92
Linea: 2      Lexema: (      Token: 75
Linea: 2      Lexema: )      Token: 76
Linea: 3      Lexema: )      Token: 76
Linea: 4      Lexema: ;      Token: 92
Linea: 4      Lexema: ;      Token: 92
Linea: 7      Lexema: ;      Token: 92
  
```

ANEXO: EJEMPLO LISTA DE TOKENS

Palabras Reservadas:

- Palabras reservadas
- Directivas procesador
- Funciones stdio.h
- Funciones stdlib.h
- Funciones string.h
- Funciones conio.h
- Funciones math.h
- otras

Simbolos:

- Aritmeticos
- Asignación, incremental, decremental
- Operadores relacionales
- Operadores Logicos
- Operadores de Agrupación
- Miscelaneos
- Caracteres de escape
- Generales

```
#region Palabras Reservadas
Palabra.Add("auto", 1);
Palabra.Add("break", 2);
Palabra.Add("case", 3);
Palabra.Add("char", 4);
Palabra.Add("const", 5);
Palabra.Add("continue", 6);
Palabra.Add("default", 7);
Palabra.Add("do", 8);
Palabra.Add("double", 9);
Palabra.Add("else", 10);
Palabra.Add("enum", 11);
Palabra.Add("extern", 12);
Palabra.Add("float", 13);
Palabra.Add("for", 14);
Palabra.Add("goto", 15);
Palabra.Add("if", 16);
Palabra.Add("int", 17);
```